

Bionic Ankle

Problem

As wars started to rage in the world, the rate of disability has gone high. More people need help on missing limbs, notably legs. The most important part of bionic leg is the ankle. Able to produce a good bionic ankle can greatly help these disabled people.

Solution

I am trying to make a bionic ankle that can relatively mimic a human ankle, and help disabled people to walk once more. Using Solidworks, I first plan out the whole structure of hardware to check out the plan's feasibility. Then, I started on building the whole project's hardware. After finishing the hardware, I then build the Arduino setup, using esp32 as the controller, a remote-control gyroscope as a sensor, and two motors as the drive force. Once all of electronics is finished, I turned to programming.

Function

The bionic ankle will use two motors to mimic the movement of a normal human foot. Using the components spring system, universal joint, and motor system, the bionic foot tries to mimic the body part's function, including absorbing shock, providing power, joints, and bones.

Ankle's Angle

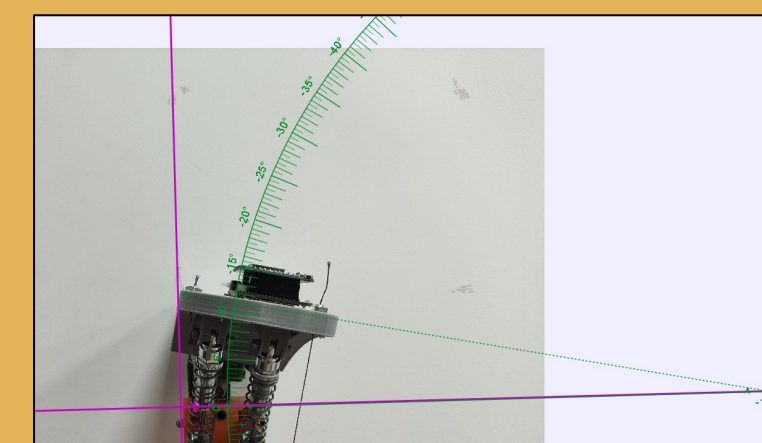


Figure 1: Angle of Eversion

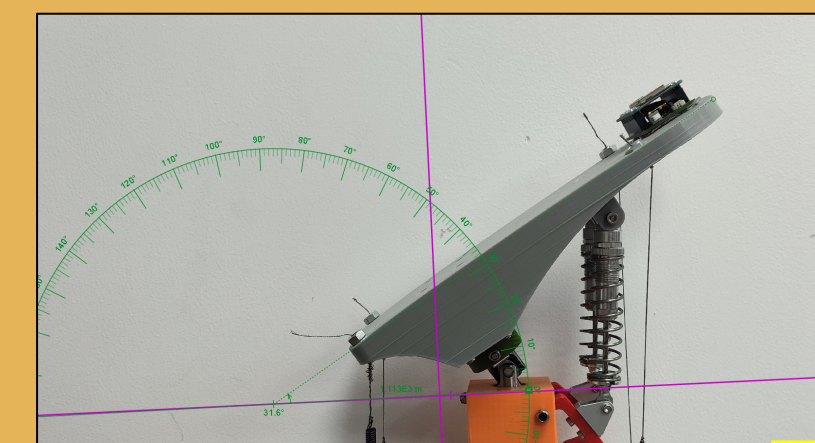


Figure 2: Angle of Planar Flexion

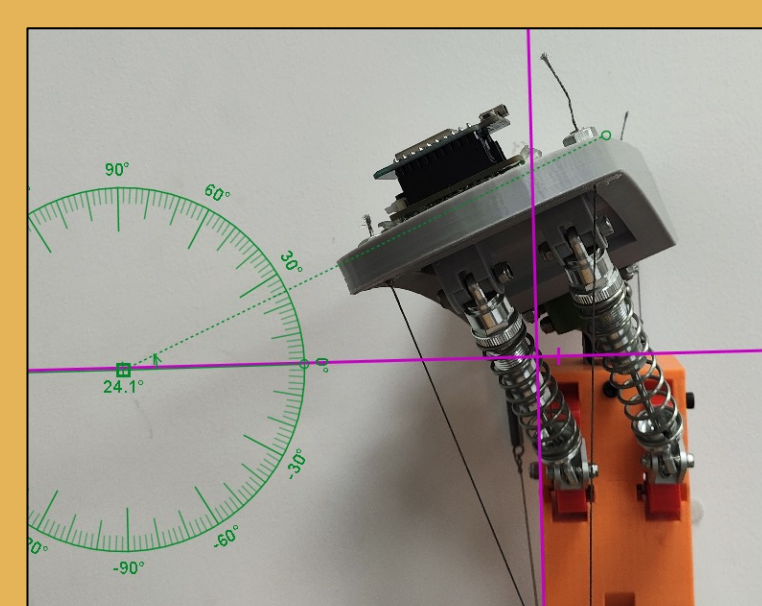


Figure 3: Angle of Inversion

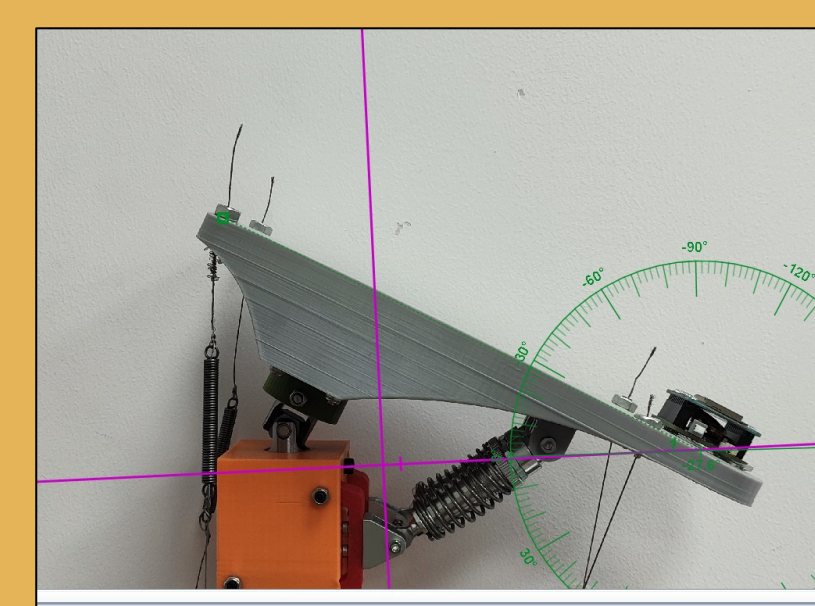


Figure 4: Angle of Dorsiflexion

Motion	Maximum angle(°)
Dorsiflexion (up)	27.6
Plantar flexion (down)	31.6
Inversion (inward)	24.0
Eversion (outward)	11.0

Table 1: The maximum angle that each movement can get

These numbers relatively matches the human's ankle movements, with the percent accuracy, except dorsiflexion, are around 26%.

Programming

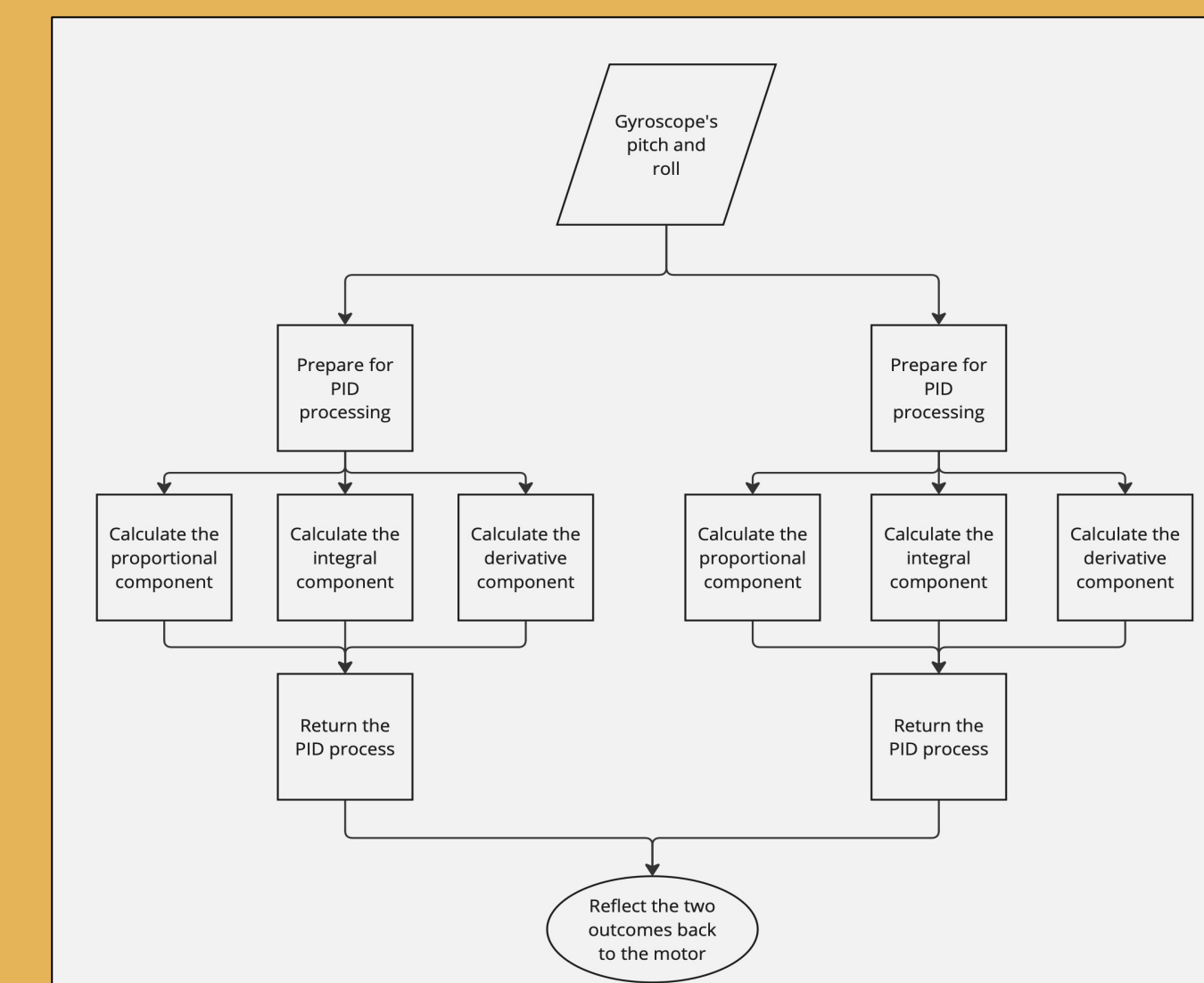


Figure 5: Software Flowchart

```
float PID_control2(float now_data2, float set_data2)
{
    float err2 = set_data2 - now_data2;
    err_all2 = err2 + err_all2;
    if (err_all2 > 1800) err_all2 = 1800;
    if (err_all2 < -1800) err_all2 = -1800;
    float P_OUT2 = p_kp * err2;
    float I_OUT2 = p_ki * err_all2;
    float D_OUT2 = p_kd * (err2 - P_last_err);
    P_last_err = err2;
    float all_out2 = P_OUT2 + I_OUT2 + D_OUT2;
    if (all_out2 > 1200) all_out2 = 1200;
    if (all_out2 < -1200) all_out2 = -1200;
    return all_out2;
}

void PID_total(float roll, float pitch, float set_pitch, float set_roll)
{
    roll_PID_angle = PID_control2(roll, set_roll);
    pitch_PID_angle = PID_control2(pitch, set_pitch);
    angle_increase_send(roll_PID_angle - pitch_PID_angle); delay(10);
    angle_increase_send(roll_PID_angle - pitch_PID_angle); delay(5);
}
```

Figure 6: PID controller code

The programming of the bionic ankle relies on a 2 variable PID control. The gyroscope will send its angle roll and pitch (one describes the rotation angle around the x axis, and one describes the rotation angle around the y axis). I then separately use PID controller on the two elements. Then, combine the two outcomes and send it to the two motors, letting them increase or decrease their angle.

Background

The idea of a bionic ankle has existed for ages, but most bionic ankle focuses on improving only the gait pattern of the bionic ankle and shock absorption.



Figure 7: Ottobock Meridium's Bionic Leg

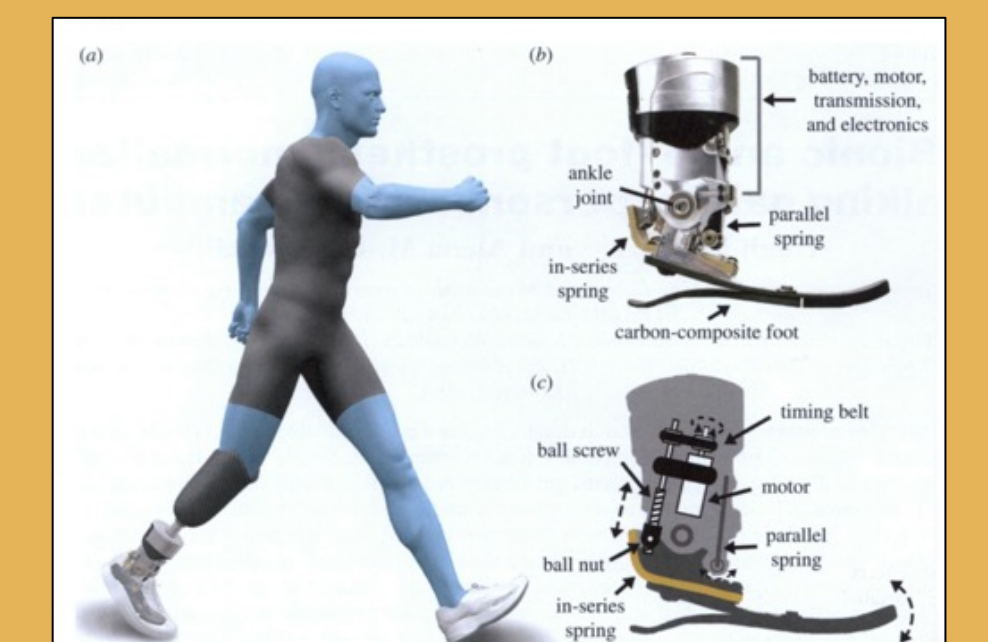


Figure 8: BiOM's bionic ankle

Innovation

1. Added inversion and eversion movements
2. Used a better spring system to mimic human's ankle function
3. Used ADAMS to model the kinematics for optimized lengths

Final Product

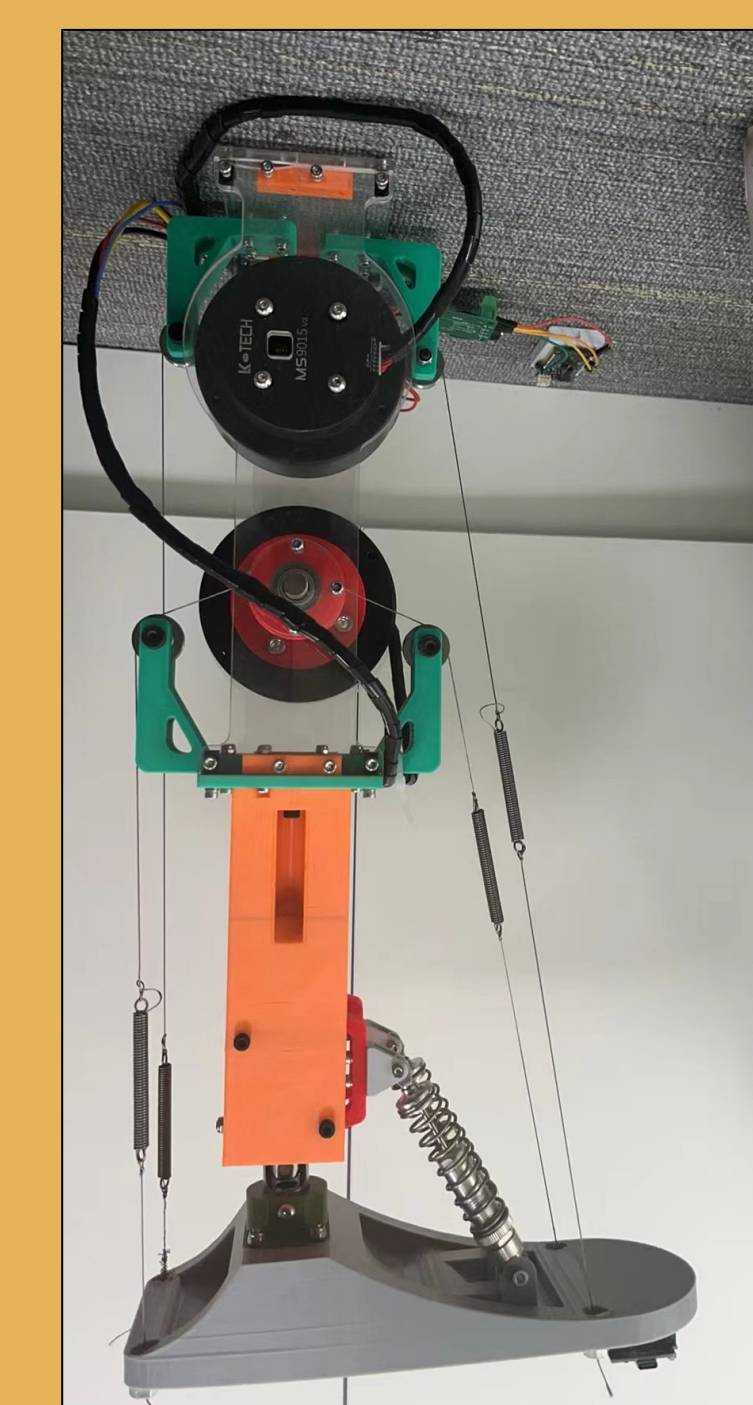


Figure 12: Actual Work

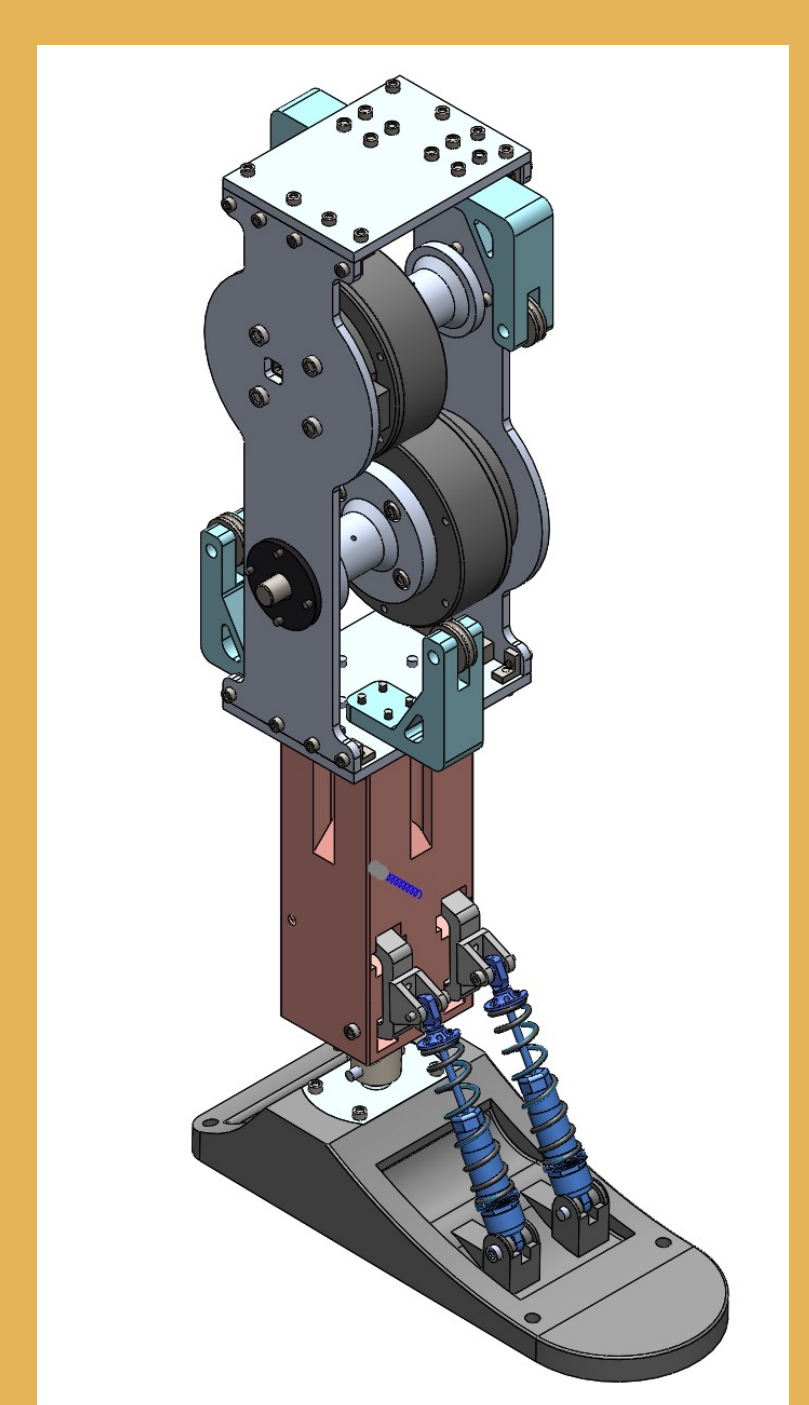


Figure 13: Solidworks Model

The Components

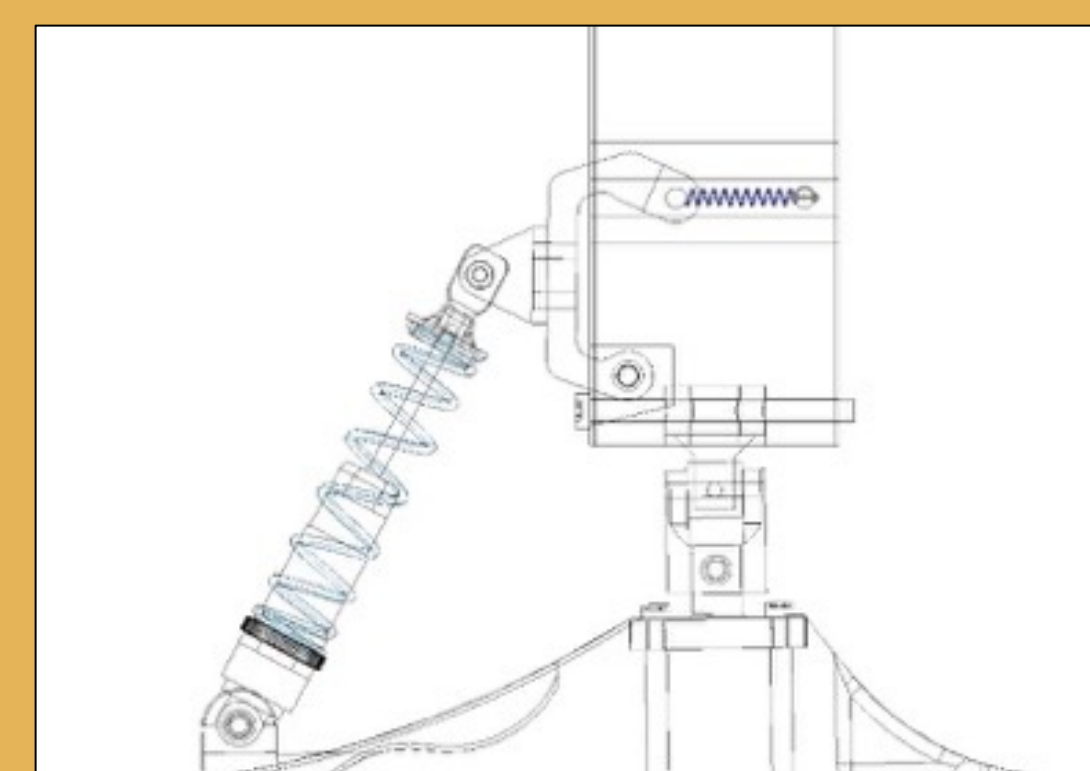


Figure 9: The spring system

Spring System: limit the movement of the foot, absorbs shock

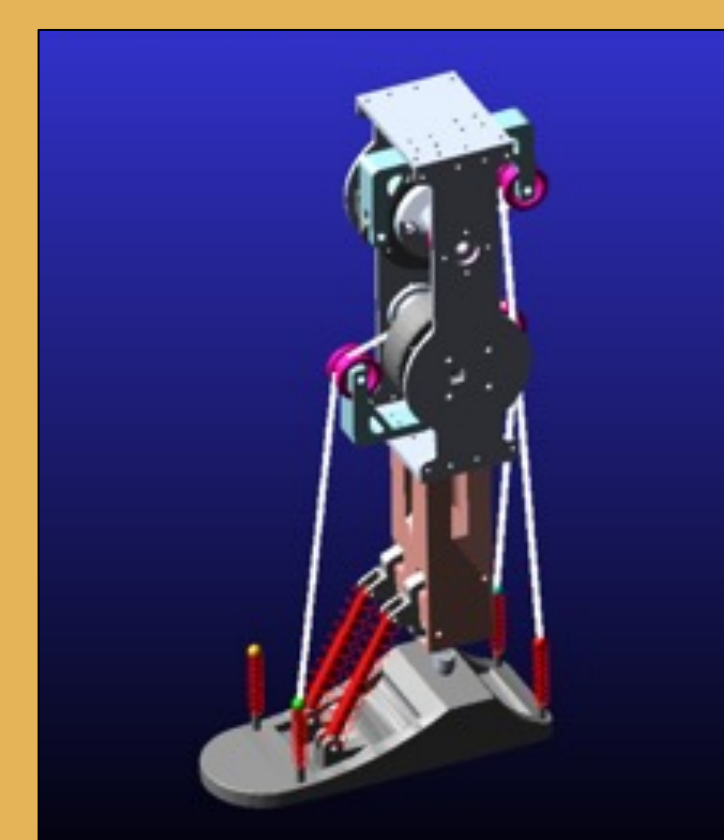


Figure 10: The Motor system

Motor System: Drive force of the bionic foot

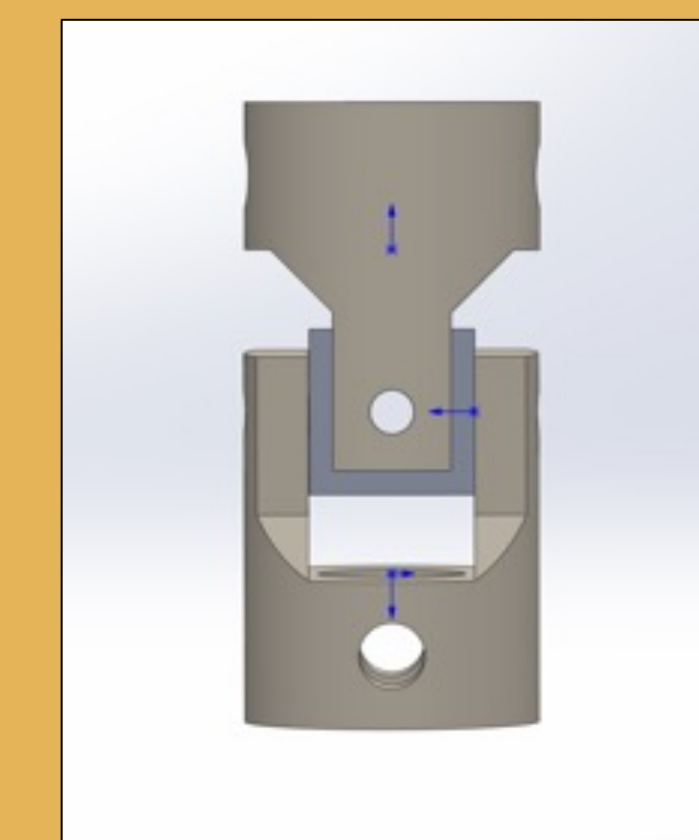


Figure 11: Universal Joint

Universal Joint: Allows the foot to move 2 dimensionally